

NumPy入門

配列操作・演算・速度比較

rkskmt

1

NumPyとは

- NumPyは数値計算のデファクトスタンダード
 - 普通、数値計算やデータ処理にlistは使わない
- 内部的にはC言語で実装されている
 - 通常のリストに比べて非常に高速

2

インストールと使用

ターミナルからインストール

```
Shell
$ pip install numpy
```

冒頭でimportして使用（ただしnpというエイリアス指定が一般的）

```
Code
1 import numpy as np
```

① importの as とは？

import numpy as np の **as np** は **別名（エイリアス）** をつける構文

```
Code
1 import numpy          # 使用時にnumpy.array(...) と書く
2 import numpy as np    # np.array(...) でよいので短い
```

- **注：** **np** の部分は何でもよく別の名前も使用可能
 - ただし、NumPyは **np** が非常に一般的

3

NumPy配列 (ndarray)

- 数値のリストっぽいものをnpに渡すとndarrayになる
- **ndarray** は **list** によく似たNumPy用の配列型
 - 先頭の **nd** は多次元配列 (**N-Dimensional Array**) の略
- NumPyはndarrayを効率よく処理するためのライブラリ

Code

```
1 import numpy as np
2
3 arr = np.array([1, 2, 3, 4]) # 1-4のndarray
4
5 # 以下でarrを数値計算 (例: 平均・分散 etc.)
```

① ノート

np.ndarray() ではなく **np.array()** と書くことに注意

4

基本的な初期化

- リストを **np.array()** に渡すと、NumPy配列になる

Code

```
1 import numpy as np
2
3 li = [1, 2, 3, 4]
4 arr = np.array(li)
5
6 arr = np.array([1, 2, 3, 4]) # 同じ
7 arr = np.array(range(1, 5)) # 同じ。実際には後述のarangeがよく使われる
```

⚠ 変数名を **list** にしない

list = [1, 2, 3] と書くと**組み込みの list 型が上書き**され、以降
list(range(5)) などが謎のエラーになります。**li** や **numbers** を使いましょう。

5

初期化（ヘルパー関数）

- 下記がよく使う数列は、ヘルパー関数で一撃生成
 - 特に **random** 系は強力
- テスト用データを作るときに非常によく使われる

Code

```
1 import numpy as np
2
3 np.zeros(3)          # 0.0が3つの要素で初期化。 [0. 0. 0. ]
4 np.ones(5)           # 1.0が5つの要素で初期化。 [1. 1. 1. 1. 1.]
5 np.full(4, 7)        # 7が4つの要素で初期化。 [7 7 7 7]
6
7 # arrangeではなくarange ("a range" = ある範囲)
8 np.arange(4)          # [0 1 2 3]
9 np.arange(1, 5)       # [1 2 3 4]
10 np.arange(1, 10, 2)  # [1 3 5 7 9]
11
12 np.random.rand(3)     # [0.0, 1.0) の一様乱数 3要素
13 np.random.randint(10) # 0~9のランダムな整数
14 np.random.randint(1, 7, 3) # 1~6のランダムな整数 3要素
15
16 arr = np.arange(10)
17 np.random.shuffle(arr) # 配列をランダムに並べ替え（破壊的）
```

6

要素へのアクセス

- アクセス方法はlistと同じ
- スライスにも対応

Code

```
1 import numpy as np
2
3 arr = np.array([1, 2, 3, 4])
4
5 print(arr[0])        # 先頭の要素 (1)
6 print(arr[1])        # 2番目の要素 (2)
7 print(arr[-1])       # 最後の要素 (4)
8 arr[0] = 10          # 1番目の要素を10に変更
9
10 # スライス
11 print(arr[1:3])      # 2番目から3番目まで ([2, 3])
12 print(arr[:2])       # 先頭から2番目まで ([1, 2])
13 print(arr[2:])       # 3番目から最後まで ([3, 4])
```

7

型

- ndarray全体は **numpy.ndarray**
- 各要素は代入時のリテラルによって決まる
 - intなら **numpy.int64**
 - floatなら **numpy.float64**
- 異なる型は混ぜられない (intとfloatを混ぜると **全部floatに統一** される)

Code

```
1 import numpy as np
2
3 arr = np.array([1, 2, 3, 4])
4 print(type(arr))    # <class 'numpy.ndarray'>
5 print(type(arr[0])) # <class 'numpy.int64'>
6
7 arr2 = np.array([1.0, 2.0, 3.0, 4.0])
8 print(type(arr2))   # <class 'numpy.ndarray'>
9 print(type(arr2[0])) # <class 'numpy.float64'>
```

8

強力な演算

- Listと違い、各要素に対する演算が可能
 - for文が不要なため非常に高速

Code

```
1 import numpy as np
2
3 my_array = np.array([1, 2, 3])
4 print(my_array * 2) # [2 4 6] ← 要素ごとの計算
5
6 # listに * 2 を使うと...
7 my_list = [1, 2, 3]
8 print(my_list * 2)  # [1, 2, 3, 1, 2, 3] ← 演算ではなく、2つのリストのconcat
9
10 # listで同じことをするにはfor文や内包表記が必要
11 print([x * 2 for x in my_list]) # [2, 4, 6]
```

9

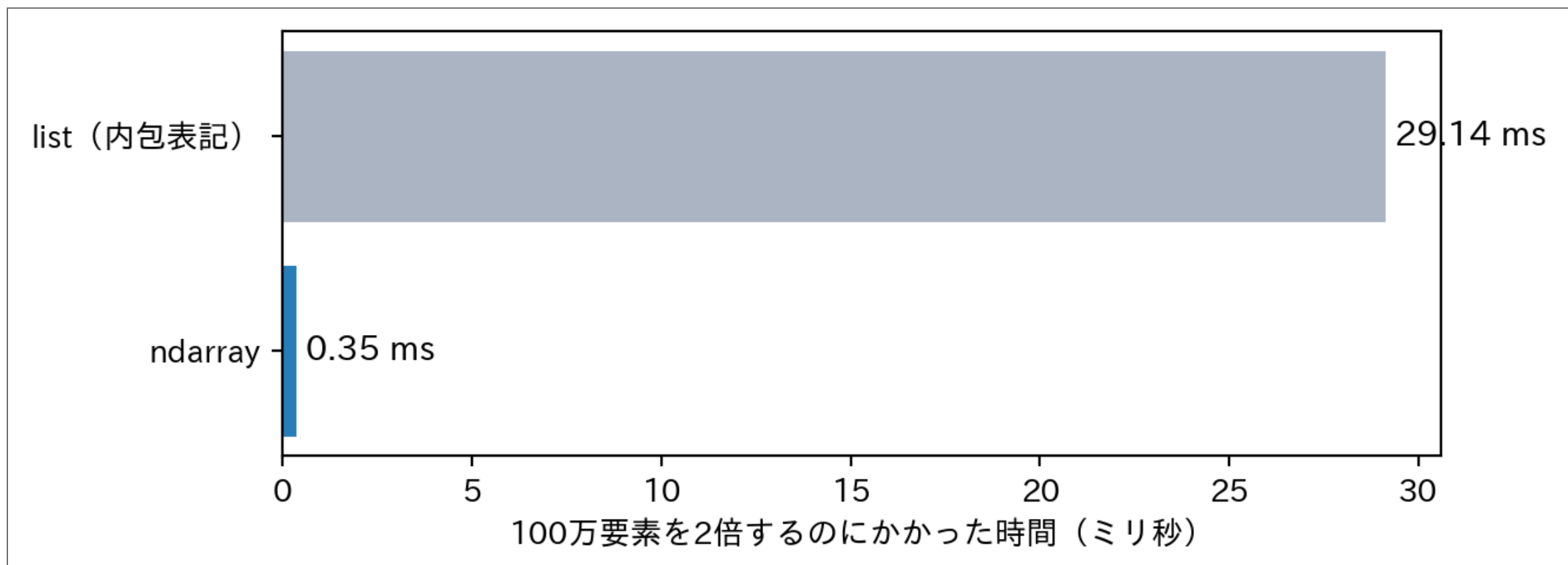
本当に速いの？ — 100万要素で実測

- 「for文が不要なため非常に高速」 — 何倍くらい違うと思う？

▶ 計測コードを表示

Result

```
list (内包表記) : 29.1 ミリ秒
ndarray:         0.35 ミリ秒
→ 約 83 倍速い
```



- C言語で実装されたループが中で回るため、**桁違いに速い**
- データが大きいほど差が開く — 「数値計算にlistは使わない」のはこれが理由

10

その他（長さやアクセス方法）

- listと同じくlenもある
→ ただし、sizeを使うことが多い
- for文はlistと同じように記述可能

Code

```
1 import numpy as np
2
3 arr = np.array([4, 5, 6])
4
5 print(len(arr)) # 長さ (listと同じ)
6 print(arr.size) # 普通はこちらを使う
7
8 # for文もlistと同じように使える
9 for v in arr:
10     print(v)
11
12 # 差分 (数値微分)
13 arr = np.array([0, 2, 5, 3, 1, 4]) # 時系列の信号を想定
14 diff = np.zeros(len(arr) - 1) # 結果格納用のndarray
15 for i in range(len(arr) - 1):
16     diff[i] = arr[i+1] - arr[i] # 微分
17 print(diff) # [ 2.  3. -2. -2.  3.]
18
19 # 実はNumPyなら1行 (こういう「定番処理の組み込み」が大量にある)
20 print(np.diff(arr)) # [ 2  3 -2 -2  3]
```

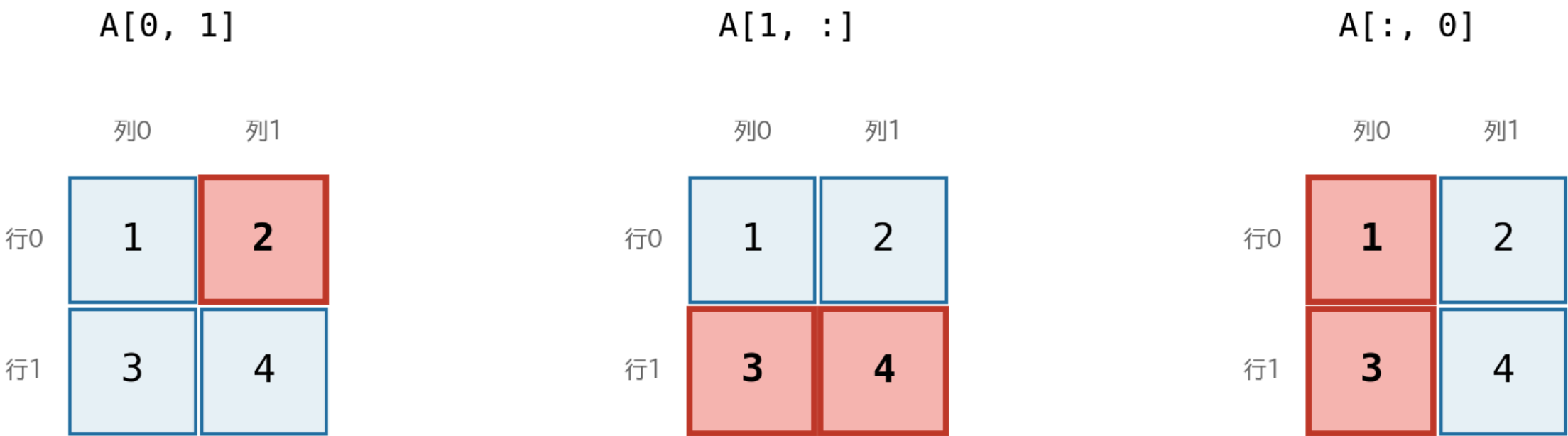
11

多次元Array

- ndarrayは多次元（**N-dimensional**）で力を発揮
→ 特に行列演算が強力

Code

```
1 import numpy as np
2
3 # 2次元配列（行列）
4 A = np.array([[1, 2], [3, 4]])
5
6 B = np.array([[5, 0],
7              [0, 5]])
8
9 # 要素へのアクセス
10 print(A[0, 1]) # 1行目2列目
11 print(A[1, :]) # 2行目全体
12 print(A[:, 0]) # 1列目全体
13
14 # 転置（transpose）
15 print(A.T)
16 print(B.T)
```



- **A[行, 列]** の順。 **:** は「その方向ぜんぶ」

統計の組み込み関数

- 統計処理の組み込み関数も多数提供されている

Code

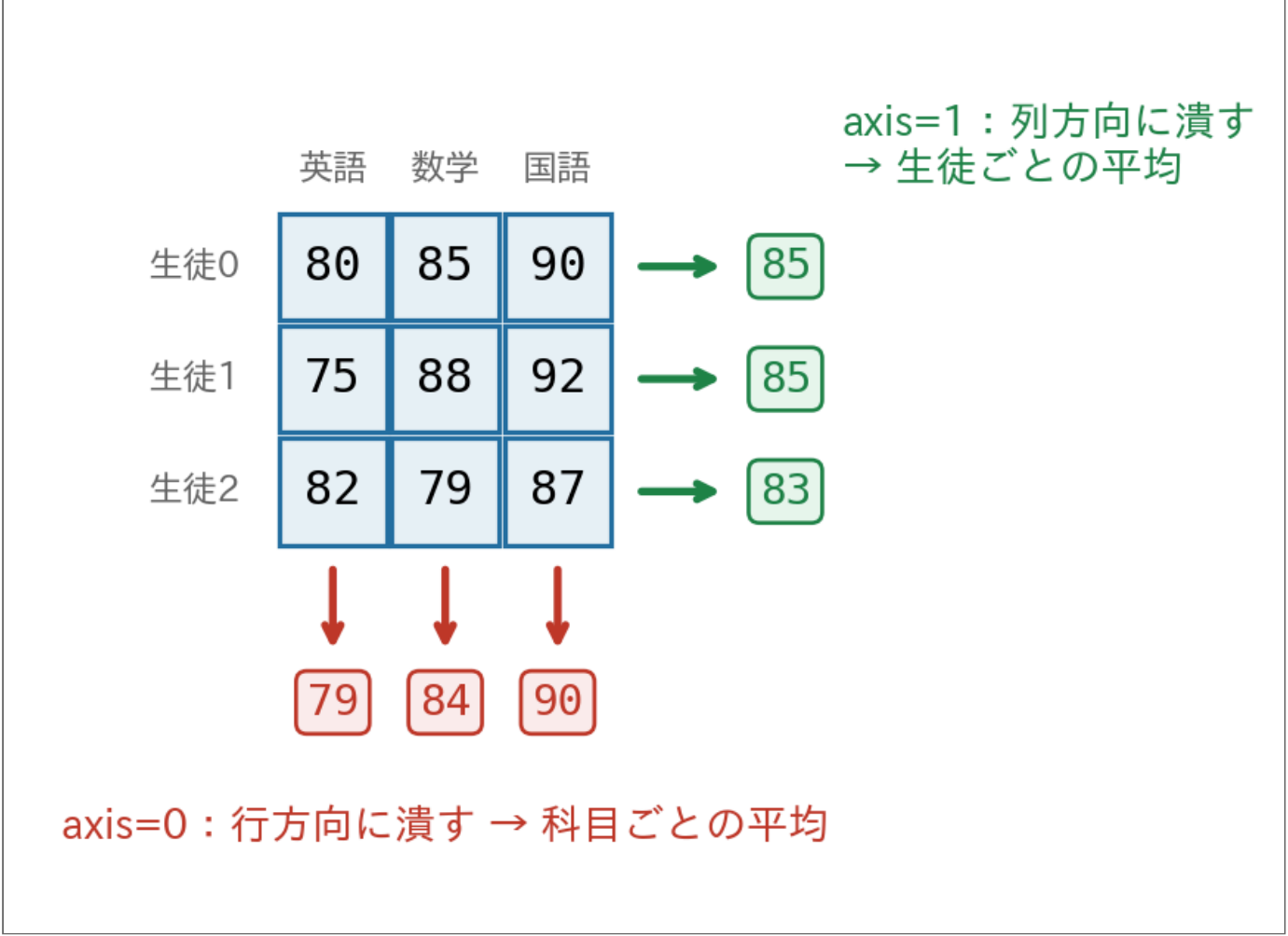
```
1 import numpy as np
2
3 scores = np.array([80, 85, 90, 75, 88, 92, 82, 79, 87])
4
5 print(np.max(scores)) # 最大値
6 print(np.min(scores)) # 最小値
7 print(np.mean(scores)) # 平均値
8 print(np.median(scores)) # 中央値
9 print(np.var(scores)) # 分散
10 print(np.std(scores)) # 標準偏差
```

多次元は axis で「潰す向き」を指定

- 2次元以上の集計は **axis=** で向きを指定する

Code

```
1 import numpy as np
2
3 # 3人の生徒の英語・数学・国語の成績
4 data = np.array([[80, 85, 90], [75, 88, 92], [82, 79, 87]])
5
6 print(np.mean(data, axis=0)) # 列ごと（各科目の平均）
7 print(np.mean(data, axis=1)) # 行ごと（各生徒の平均）
```

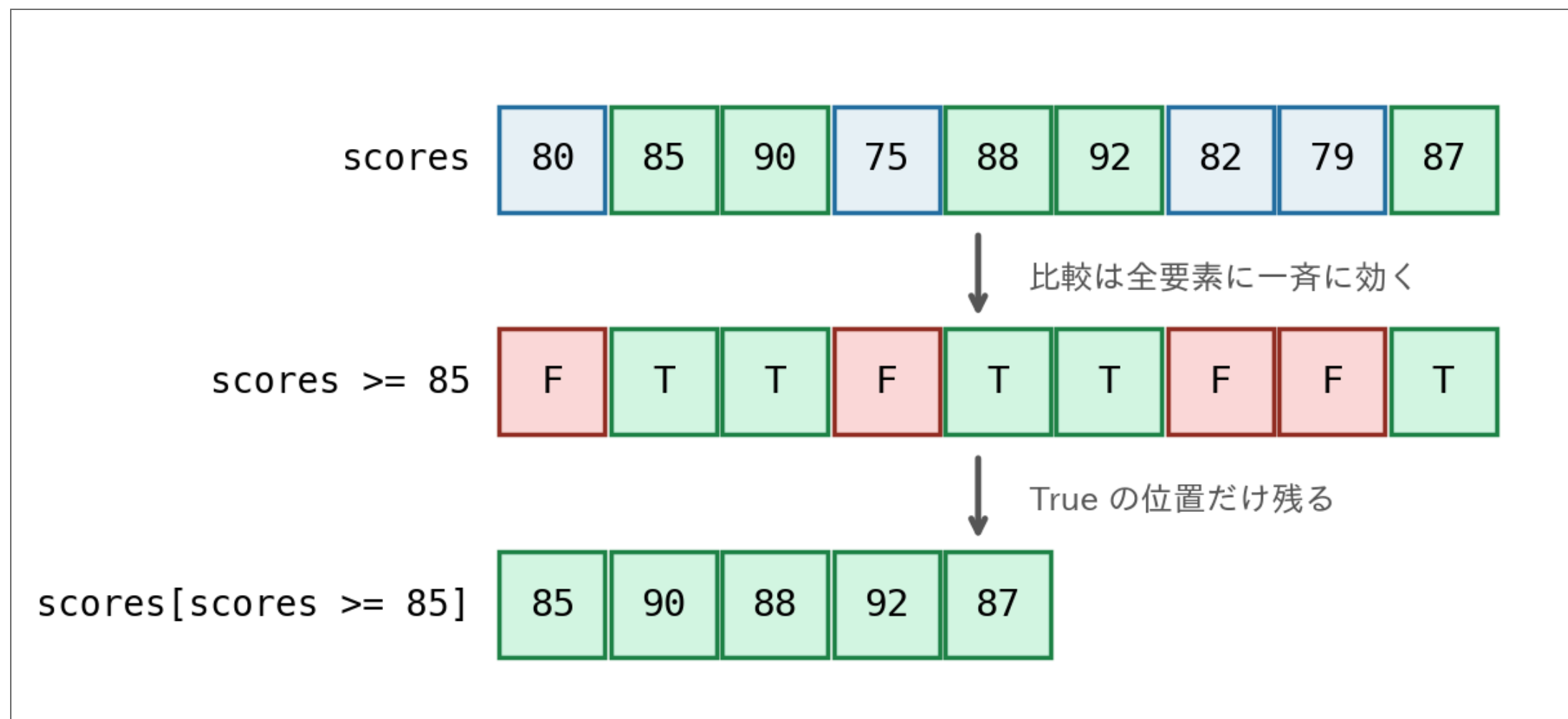


条件でフィルタする

- 比較演算も「全要素いっせいに効く
- その **True** / **False** を添字に渡すと **フィルタ** になる

Code

```
1 import numpy as np
2
3 scores = np.array([80, 85, 90, 75, 88, 92, 82, 79, 87])
4
5 print(scores >= 85)          # True/False
6 print(scores[scores >= 85])  # 85点以上のデータ
```



scores >= 85 が True/False の並びを作り、それを添字に渡すと True の位置だけ残る

15

演習：ndarrayを作って操作する



NumPyで次の5つを作り、表示せよ。

1. 「1から10までの連続した整数」から成る **ndarray**
2. その各要素を2倍にした新しい **ndarray**
3. 元の **ndarray** の各要素をランダムに並べ替えた新しい **ndarray**
4. 100以上400以下のランダムな整数 1つ
5. 0以上1未満のランダムな数 1つ

▶ 解答例を見る

16

演習：listとndarrayの型



同じ数値でも、**list** の要素とNumPy配列の要素で**型がどう変わるか**を **type()** で確かめよ。

1. **li = [10, 20, 30]** の全要素の型を **for** 文で表示
2. **li** をNumPyの配列 **arr** に変換し、要素の型を表示
3. **li = [0.1, 0.2, 0.3]** でも同様に全要素の型を表示
4. これもNumPyの配列 **arr2** に変換し、要素の型を表示

▶ 解答例を見る